



محرك الألعاب التكتيكي

محرك استراتيجية الأدوار

Turn-Based Strategy Engine

مشروع ITSE424: هندسة البرمجيات المتقدمة

الطالب

محمد البوراوي
2221810966

الطالب

عبدالعزیز اشرف الجازوي
2221808195

الطالب

علي محسن الحبشي
2221809256

إطار عمل أنماط التصميم في Java <>

مقدمة عن التطبيق

ما هو محرك الألعاب التكتيكي؟

إطار عمل بلغة **Java** لإدارة ألعاب الاستراتيجية القائمة على الأدوار، يوفر نظامًا متكاملًا لإدارة كيانات اللعبة والتحكم في تدفق الأحداث بشكل منظم وفعال. مشروع **ITSE424** يتطلب تطبيق ثلاثة أنماط تصميم على الأقل، مع التركيز على جودة الكود والتطبيق الصحيح للأنماط.

سير العمل

- 1 إضافة جنود وفرق إلى الجيش
- 2 تطبيق ترقية الأسلحة على الكيانات
- 3 التحكم في أدوار اللعبة وحالتها
- 4 تنفيذ الهجمات ومتابعة النتائج

واجهة المستخدم

- قائمة الوحدات**  عرض هيكل الجيش بشكل شجري
- سجل الأوامر**  تتبع جميع العمليات والنتائج
- أزرار التحكم**  إدارة الكيانات والترقيات والهجمات
- مؤشرات الأداء**  عرض إجمالي قوة الجيش

الأهداف

- ✓ تطبيق أنماط التصميم بشكل صحيح
- ✓ بناء نظام مرن وقابل للتوسيع
- ✓ توفير واجهة تفاعلية سهلة الاستخدام

البنية التقنية

- Swing GUI**  واجهة المستخدم الرسومية
- Collections Framework**  لإدارة كيانات اللعبة

- +Java 17**  لغة البرمجة الأساسية
- Design Patterns**  Singleton, Composite, Decorator

المكونات الرئيسية

Decorator  نظام الترقيات

GameEntity  هيكل الكيانات

GameEngine  إدارة حالة اللعبة

Singleton Pattern - المفهوم

التعريف

نمط إنشائي يضمن وجود **مثيل واحد فقط** من الفئة ويوفر نقطة وصول عالمية إليه

متى نستخدمه؟

- ◀ عند الحاجة للتحكم في الموارد المشتركة
- ◀ إدارة الحالة العامة للنظام
- ◀ تنسيق العمليات عبر النظام

المشكلات الشائعة

 الاختبار

 التسلسل

 أمان الخيوط

طرق التنفيذ <>



Eager Initialization

إنشاء المثيل عند تحميل الفئة - بسيط لكن غير فعال



Lazy Initialization

إنشاء المثيل عند الحاجة - فعال لكن غير آمن للخيوط



Double-Checked Locking

آمن للخيوط وفعال - المستخدم في مشروعنا

الخصائص الأساسية

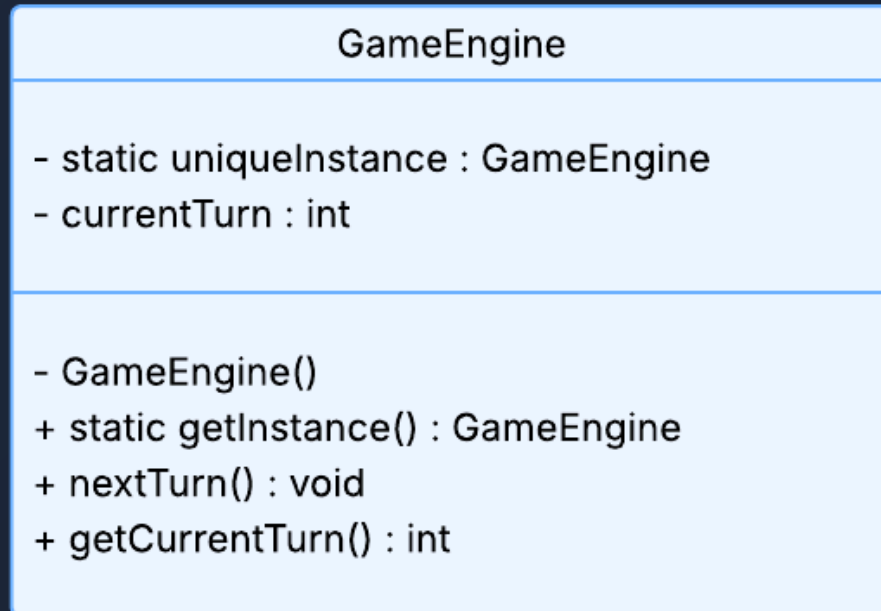
- ✓ مُنشئ خاص
- ✓ متغير ثابت خاص
- ✓ طريقة وصول عامة ثابتة
- ✓ منع الاستنساخ

Singleton Pattern

ما هو النمط؟

يضمن وجود **مثيل واحد فقط** من الفئة ويوفر نقطة وصول عالمية إليه

مخطط UML



التطبيق في GameEngine

إدارة حالة اللعبة والأدوار بشكل مركزي ✓

تنفيذ آمن للخيوط مع **Double-Checked Locking** ✓

منع إنشاء مثيلات متعددة من الفئة ✓

الفوائد

أمان الخيوط 

إدارة مركزية 

وصول عالمي 

منع التكرار 

Composite Pattern - المفهوم

التعريف

نمط هيكل يؤولف الكائنات في هياكل **شجرية** لتمثيل تسلسلات جزء-كل، مما يتيح معالجة الكائنات الفردية والمركبة بشكل موحد

متى نستخدمه؟

- ◀ عند الحاجة لمعالجة الكائنات الفردية والمركبة بشكل موحد
- ◀ تمثيل الهياكل الهرمية مثل أنظمة الملفات أو المنظمات
- ◀ إخفاء الفرق بين التراكيب والكائنات الفردية عن العميل

المكونات الرئيسية

Component

واجهة مشتركة لجميع الكائنات في التركيب

Leaf

كائن أساسي ليس له كائنات فرعية

Composite

كائن مركب يحتوي على كائنات فرعية

الفوائد

- ✓ تبسيط كود العميل
- ✓ مرونة في بناء الهياكل
- ✓ سهولة إضافة مكونات جديدة
- ✓ معالجة موحدة للكائنات

Composite Pattern

ما هو النمط؟

يؤلف الكائنات في هياكل شجرية لتمثيل تسلسلات جزء-كل

التطبيق في GameEntity

✓ **Soldier** - عقدة ورقية تمثل جندياً واحداً

✓ **Squad** - عقدة مركبة تحتوي على كائنات GameEntity

✓ واجهة موحدة لمعالجة الأفراد والمجموعات

الفوائد

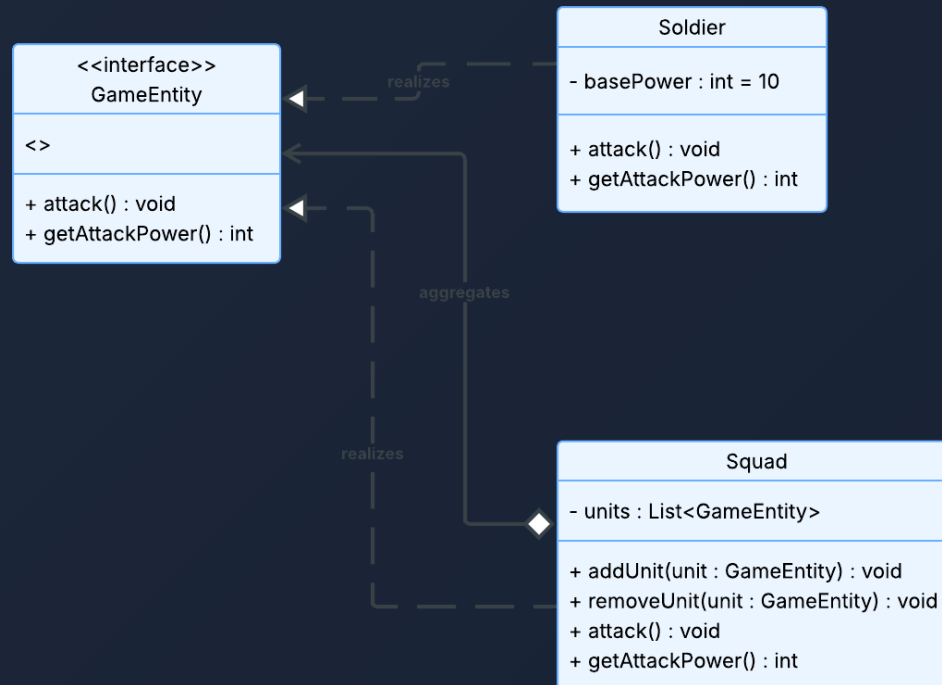
هيكल شجري مرن

معالجة موحدة

قابلية التوسع

إضافة ديناميكية

مخطط UML



Decorator Pattern - المفهوم ✂✂✂

التعريف 🧠

نمط هيكلي يضيف مسؤوليات إضافية للكائنات **ديناميكياً** دون تعديل الفئة الأصلية، كبديل مرن للوراثة

متى نستخدمه؟ ✂=

- ◀ عند الحاجة لإضافة مسؤوليات ديناميكياً وشفافية
- ◀ عندما تكون الوراثة غير عملية أو غير مرنة
- ◀ عند الرغبة في إضافة/إزالة مسؤوليات وقت التشغيل

المكونات الرئيسية 🧩

Component 🧩

واجهة مشتركة للكائنات والمزينات

Concrete Component 🏗️

الكائن الأساسي الذي يتم تزيينه

Base Decorator 📦

فئة مجردة تغلف الكائن المراد تزيينه

Concrete Decorators ✨

تضيف مسؤوليات محددة للكائن

الفوائد ⭐

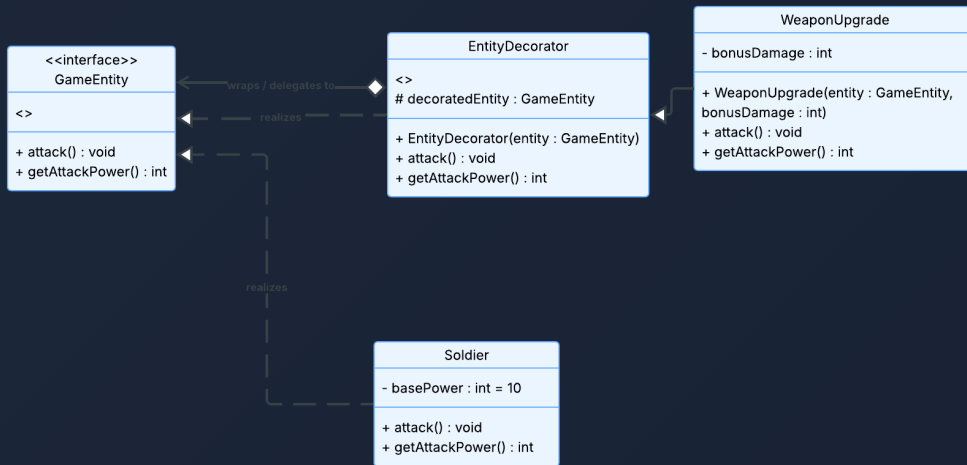
- ✓ أكثر مرونة من الوراثة
- ✓ يسمح بدمج السلوكيات
- ✓ يتبع مبدأ المسؤولية الواحدة
- ✓ تعديلات وقت التشغيل

Decorator Pattern ✨🔧

ما هو النمط؟ 🧠

يضيف مسؤوليات إضافية للكائن **ديناميكياً** دون تعديل الفئة الأصلية

مخطط UML 📐



التطبيق في WeaponUpgrade ⚙️

✓ **EntityDecorator** - الديكور الأساسي الذي يغلف الكائن

✓ **WeaponUpgrade** - ديكور محدد يضيف ضرر إضافي

✓ ترقية الأسلحة في وقت التشغيل ديناميكياً

الفوائد ⭐

⌚ تعديل وقت التشغيل

📄 توسيع مرّن

⬆️ تكوين سلوكي

🔧 بديل للوراثة

تكامّل الأنماط في التطبيق

مثال على سير العمل

+ إضافة كيانات باستخدام Composite

إنشاء جنود وفرق وتنظيمها في هيكل شجري

↑ تطبيق ترقيات باستخدام Decorator

إضافة ترقيات الأسلحة للكيانات المحددة

» تقديم الأدوار باستخدام Singleton

التحكم في تقدم اللعبة وحالتها

⚡ تنفيذ الهجمات عبر Composite

معالجة متعددة الأشكال لجميع الكيانات

كيف تعمل الأنماط معًا؟

Singleton يدير الحالة العامة للعبة والأدوار

Composite ينظم كيانات اللعبة في هيكل شجري

Decorator يعزز قدرات الكيانات ديناميكياً

التكامل بين الأنماط

✓ كل نمط يكمل الآخر في بناء النظام

✓ فصل المسؤوليات يسهل الصيانة والتطوير

✓ مرونة عالية في إضافة ميزات جديدة

القوة المشتركة

تكامّل الأنماط الثلاثة يخلق نظامًا **مرنًا وقابلًا للتوسيع**، حيث يمكن إضافة ميزات جديدة دون تعديل الكود الحالي، مع الحفاظ على بنية نظيفة وسهولة الصيانة.

الخلاصة

Decorator

- ترقية ديناميكية
- بديل للوراثة
- تكوين وقت التشغيل

Composite

- تكوين جيش مرن
- هيكل شجري
- معالجة موحدة

Singleton

- إدارة مركزية للحالة
- مثيل واحد فقط
- أمان الخيوط

التكامل بين الأنماط

تعمل الأنماط الثلاثة معاً لإنشاء **محرك ألعاب مرن وقابل للتوسيع والصيانة**، حيث يوفر Singleton إدارة مركزية، ويبنى Composite هياكل معقدة، ويتيح Decorator إضافة ميزات ديناميكياً 

فريق العمل

محمد البوراوي 
2221810966

عبدالعزیز اشرف الجازوي 
2221808195

علي محسن الحبشي 
2221809256